

30/05/2024



Final Report Echo

University of York

Group 1

Dylan Soco

+ Marcy Williams

+ Tyler Noah Green

+ Jacob Robertson

+ Benedict Francis

Contents

pg. 3	Product Concept
pg. 5	Description of Entire System
pg. 7	Analysis of Effectiveness
pg. 8	Future Plans
pg. 10	Individual Contribution
pg. 11	Technical Detail
pg. 14	Reflective Summary
pg. 13	Peer Review
pg. 16	Appendix

Product Concept

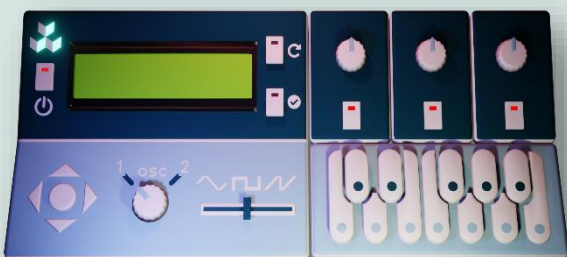
Our company, Illogitech Inc., was conceived with one clear goal in mind:
to innovate music technology education.

As such, we will heavily market towards schools and other educational establishments to innovate the overlooked aspect of music education. We want to make learning interactive, intuitive, and most importantly, fun.

With this in mind, our product will take the form of an audio-based games console. Due to the limitation of the LCD screen, we were inspired by early handheld consoles, such as the Gameboy, or even Tamagotchis. These designs are very simple, and immediately intuitive to today's generation of students, so we wanted to modernise these designs, and give our console more of an identity.



As for the game aspect, we looked for overlooked portions of the music curriculum in schools. We saw a distinct lack of any substantive music technology education, and how it can be used in tandem with modern music. Keeping in line with the fact we want to explore an aspect of music technology that is fun to play with and educational, we decided our game should explore audio synthesis.



With our concept finalised, we focused on our work breakdown to ensure our vision will come true. We split the development into 3 main verticals: PD, Software and Hardware.

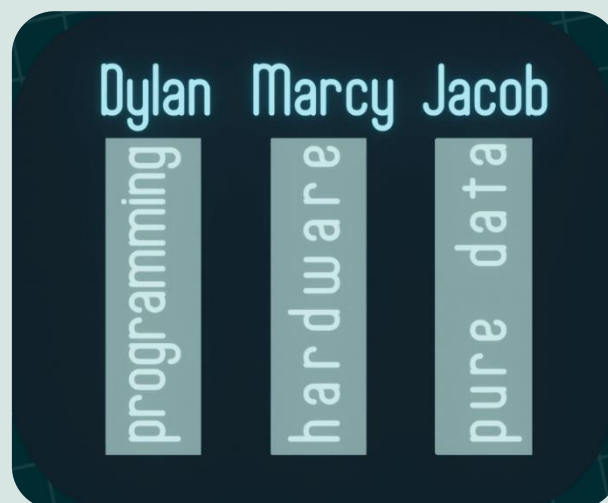
Our team of 5 discussed our strengths and weaknesses, and delegated roles between us. We had 3 main roles: **Project Lead**, **Creative Lead** and **Secretary**, which were held by me, Marcy, and Tyler respectively. We had sub-roles as well, which focused on three verticals in the project: programming, hardware, and PD.

Some of these rules include:

- Concept Artist
- PD Designer
- Interface Programmer
- Logic Programmer
- Quality Tester

Dylan	Project lead, lead programmer, secretary
Marcy	Creative lead, hardware engineer, concept artist
Jacob	PD designer
Tyler	3D Modeller
Ben	Script writer, storyboard artist

To keep up with our schedule for deliverables and other reports, we laid out our expectations for work and progress in a Gantt chart (*see Appendix A*). This lays out our work schedule for the next 16 weeks, including deadlines for deliverables, and soft deadline for when we expect certain milestones in the product's creation to be reached.



Description of Entire System

To set up the game, all that is needed is the physical console, and a computer with a USB input, and PD installed with our patch.

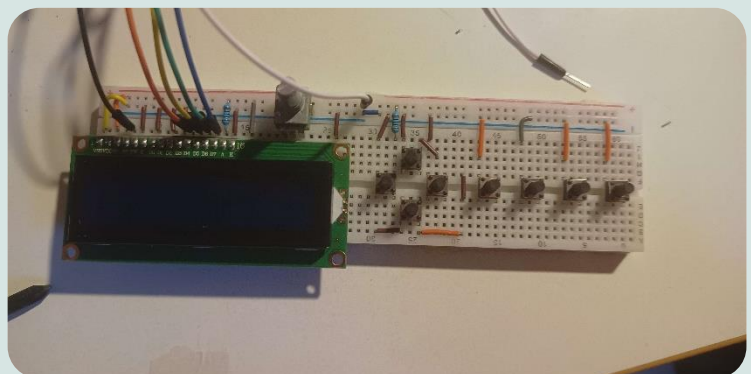
When the game starts, the console will play a randomised synthesised sound. It's then the player's goal to match that sound using the parameters provided, with the console giving live feedback on how the sound is changing. If needed, the player can press the Repeat Sound button if they want to hear the sound again. When done, the player can press the Confirm Choice button to check their recreation with the random sound, and then a percentage score, based on how many parameters they got correct.

The game offers 9 different possible answers, ranging from 1-3 parameters changing, which are randomly chosen when the game starts. The changeable parameters may include:

- wave shape
- unison
- chorus
- reverb
- filters

Currently, we have 6 buttons on our console, which can control the entire game. There are:

- 4 directional buttons.
- 1 Replay Sound button.
- 1 Confirm Sound button.

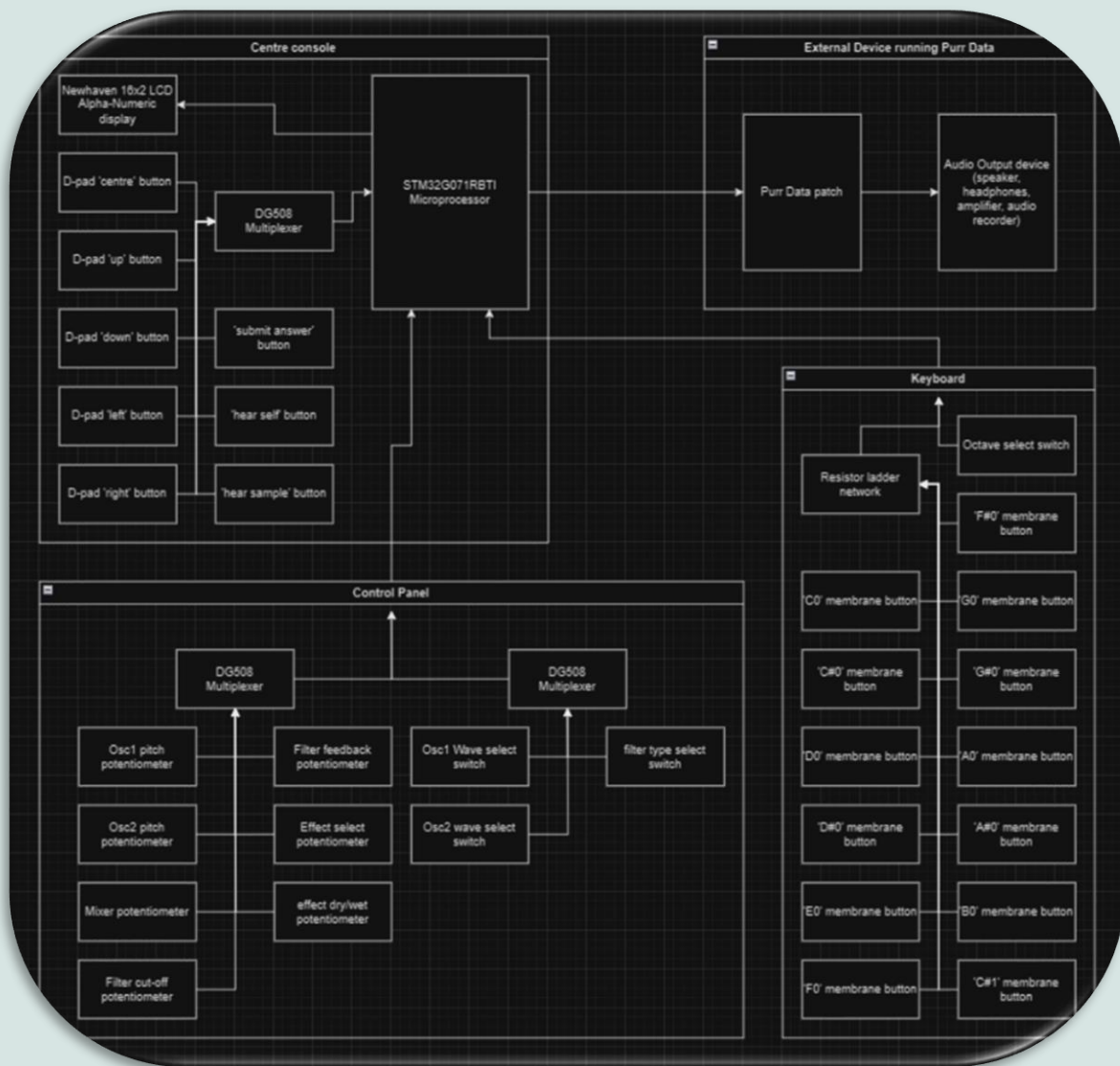


In terms of specification, our console will be built using a Nucleo board, using a STM32G071RBTI microprocessor that will be running the game code on it locally.

It will be powered through a USB connection to a computer, which will also act as the bus that will communicate with Pure Data (PD) using our own custom MIDI-esque code. A PD patch downloaded on the computer will also run our games audio-engine, and its logic, using the computer's sound card and speakers to play the audio out loud.

Our button inputs are tactile computer switches, which adds a level of tactility to our inputs. The LCD screen is a generic HD44780 16x2 character LCD Display.

Finally, the case is printed in an organic PLA plastic, which is environmentally friendly, but robust enough for our product.



Analysis of Effectiveness

Our initial goal of getting a functioning product that plays a sound and the user must match it was achieved with our prototype. We were able to implement 3 oscillator types, then a unison, filter, and reverb parameter, which can give us 36 unique combinations for possible answers. These parameters are easy to understand and recognise, so our first prototype will be easily digestible for students unfamiliar with additive synthesis.

The innovative nature of our product ensures our goal of spreading music technology education is met. Synthesisers in general are skimmed over in many music curriculums in the UK and US, so introducing our product to the market will be effective in developing these areas.

However, there are some shortcomings when it comes to our prototype. As we only have 9 premade answers, our game lacks replay value. This can easily be fixed, however, by adding more parameters as detailed in the *Future Plans* chapter.

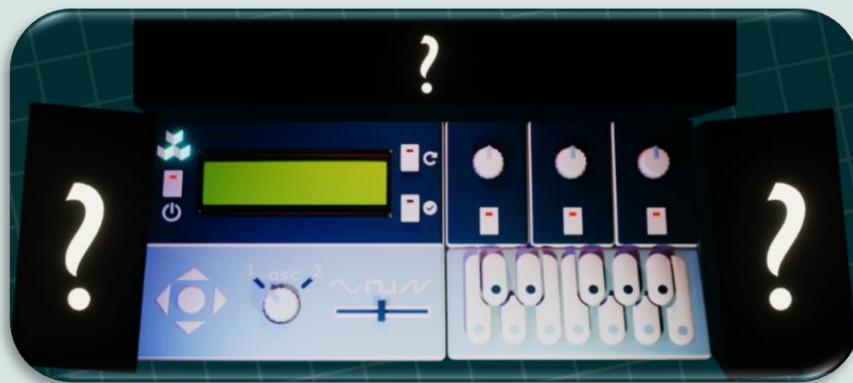
In terms of accessibility, we believe we've made a product that is accessible to blind users, as the layout of our product is intuitive and recognisable without the need of sight. The Echo's inputs are responsive and have obvious changes in sound depending on the input, so while the LCD screen is a useful visual aid, it's not necessary to play our game. Unfortunately, as our game is inherently tied to music, the LCD screen doesn't convey enough information to cater to deaf users.

Our product is predicted to be cost-effective too. We estimate each console would cost £40 per unit; however, we can cut costs by buying in bulk. This means further accessibility to our product for schools and other educational institutions.

Future Plans

Our future plans aim to have our product more closely align with the vision in our Tender Presentation Video. Most notably, we want to focus on implementing a modular design, which can allow further expansions to our console if we desire. Some modules include:

- a one octave wide keyboard, which can be wider if more keyboard modules are added.
- physical modules for effects parameters
- a second oscillator switch, which adds another layer of depth to possible sounds that can be made.



In terms of the technical aspect of how the modular design could be achieved, we could either have a magnetic system that can snap modules together, and transmit data through that connection as well, or have USB connections between each module. The latter would be easier to implement, however the former is more user-friendly.

These potential additions would make a potential free-play mode more flexible, and closer to a traditional synthesiser. Having a wide range of modules that can come with the Echo also widens the range of accessibility when marketing to schools and other educational institutions, as they can choose how many expansions they want to have.

Expanding on the free-play mode would be beneficial to the effectiveness of our product as well. Currently, our synthesiser operates off subtractive synthesis, however we could add different modes to have different types of synthesis, such as additive or frequency modulation (FM) synthesis. Most traditional synthesisers tend to be expensive, such as the OP-1 which is similar in design to ours, and digital synthesisers lack the tactility and physicality when creating sounds with a synth. Offering a cheap taster of how a synthesiser works to the market would be a fantastic tool for beginners.

As for changes in our current design, having the game run without a computer would be our first goal. This could be done with having a Raspberry Pi running PD locally, and then we can attach speakers in terms of hardware to play the music. Another addition we could implement would be an upgrade to our display, as some ideas like having a waveform give a more visual representation of what sound is being created would help convey information to deaf users. An upgrade in display would also help with the user interface, as we don't have an efficient way of displaying if the D-pad buttons are controlling the variable select, or toggle select.

Upon that, as our entire casing and keycaps are 3D printed, which took around 20 hours in total to print everything. As this is very costly in terms of time, if we were to expand, we would likely turn to resin printing, as this is easier to accomplish on a larger scale.

Individual Contribution

At the start of development, we discussed our individual strengths and weaknesses. I was appointed as the main role of Project Lead, which saw me recording company meeting notes as a secretary, keeping track of progress, and general organisation of our team. As for development, I had the most experience with coding, so I was also appointed Lead Programmer as well.

Below are my individual contributions for each role:

Project Lead

- Set-up communication channels and file sharing, including a Google Drive, GitHub repository and Discord server.
- Created a Check-In page spreadsheet and Gantt Chart to keep track of weekly progress, task assignments, and task details.
- Created a Risk Assessment to track potential dangers in development, management, and functionality.
- Organised and schedules weekly meetings.
- Recorded meeting minutes for every weekly meeting and communicating with our supervisor via email.

Lead Programmer

- Created an algorithm for the game logic.
- Programmed all the code for the working prototype, including game logic, menus and communications with PD and the LCD screen.

Miscellaneous

- 3D modelled initial and final concept models for visualisation.
- Video edited, animated, sound-designed, and produced visuals for Tender Presentation Video, as well as contributing to voice-overs.
- 3D printed finished casing.

Technical Detail

My technical contributions were seen in my contributions in the programming in the game.

Before then, I set-up my coding workflow:

- IDE: *Keil Studio*
- Language: *C++*
- Repository: *GitHub*

First, I figured out the logic for how the menu system must work, using flags:

1. Detect when a button is pressed.
2. Add 1 to the current flat. If it's outside the bounds, loop around.
3. Change the player's stored answer.
4. Display the current flag.
5. Go back to 1.

To implement this, I first designed a class called *answerBank* that would hold the stored value for each variable, for both the random answer provided by the console, and the inputted answer provided by the player.

```
class inputBank
{
    int variable[4];

    // default answer is all off
    inputBank(int answerWave = 0, int answerFilter = 0, int answerUnison = 0, int answerReverb = 0)
    {
        variable[0] = answerWave;
        variable[1] = answerFilter;
        variable[2] = answerUnison;
        variable[3] = answerReverb;
    }
}
```

Then, I needed global flags and arrays that can be referenced by every thread, to ensure all parts of the system know what the menu is currently showing.

- *layerSelect* tells us if the display should be changing the top layer (variables), or the bottom layer (values)
- *variableSelect* tells us what variable is currently being changed.
- *toggleToNum* is an array which takes the value of the current variable, and translates it to the MIDI data in Pure Data
- *toggleToString* and *variableToString* is another array which takes the value of the current variable and translates it into a string for the LCD screen.

For the button inputs, I used binary semaphores for each of the 6 buttons. which would detect when a button is pressed, and then communicate with each thread that a change has happened. Depending on what value *layerSelect* is on, pressing the left or right buttons will increment/decrease the value of the current flag.

```
if(layerSelect == 1)
{
    switch(variableSelect)
    {
        case 0: //osc
            finalAnswer.variable[0] =
(finalAnswer.variable[0]+offset)%3;
            //increment, with wraparound
            displayLCD(toggleToString[0]
[finalAnswer.variable[0]]);
            //display on LCD screen
            break;

        //...
```

Then I needed the logic on what would happen when a button is pressed. Up and down button logic was easy, as it switched the flag *layerSelect* between 1 and 0 when either of them was pressed. The left and right buttons relied on what *layerSelect* was, so I had to implement a further if statement to check. As for the variable select, each one had 3 possible states it could be in. To limit the number of if-else statements, I used a switch case, which ran logic that would first increment the flag, and then display the updated variable on the LCD screen.

I then implemented the replay and confirm buttons, both of which run in its own thread. When the replay button is pressed, the player's sound is paused, and then the answer sound is played. After a short pause, the player's sound is played again. When the confirm button is pressed, the answer sound is played, and then the player's sound is played. The display then shows the number of parameters correct, using logic in the class that compares each element in the inputBank class.

```
int getScore(const inputBank& inputArray)
{
    int runningTotal = 0;
    const int arraySize = 4;

    for(int i = 0; i < arraySize; ++i)
    {
        if(inputArray.variable[i] == variable[i])
        {
            runningTotal = runningTotal + 1;
        }
    }
}
```

With the infrastructure implemented, I put it all together. When the game is turned on, a random premade answer is picked from an array, using *cstdlib*. After starting all the button threads and semaphores, the game plays the target sound, and then after a delay, starts playing the default sound the player can now change.

Reflective Summary

In practice, the roles I had meant I was very hands-on with many aspects of the project, including solely managing the group, and contributing a major part of the project's functionality. Next time, I would delegate more responsibilities to other members of the group, so the workload could be shared more effectively.

Even with our weekly meetings and Check-In Page (*see Appendix B*), and Gantt chart, our collective time management started to falter a few weeks into the project, when a lot of work seemed to congregate nearer to the deadlines for our deliverables. Although we had a solid framework for how our time should be spent, we didn't have any safeguards to ensure our on-going company goals were met. This resulted in us having to simplify the prototype to meet our final deadline, however, it was still able to function as intended.

Despite this, we were able to set up effective communication and progress reports when nearing the end of our deadlines. Discord was an effective tool for us to communicate, as we were able to quickly discuss changes and plans in real-time, and store screenshots and pictures of progress. However, this proved difficult to find previous messages or announcements, as a lot of them

As for personal progress, I was able to consistently produce detailed meeting reports every week, which documented our progress, and goals for next week. This also meant actively updating the Weekly Check-in to track what tasks people were working on for the week, and who was working on them. In terms of the programming, I also unfortunately had a lot of the work congregate nearer the deadlines. Next time, I should spread the work out more evenly, and delegate more of it to other members of the group, so the work can be done in a more consistent and scheduled manner.

More specifically in terms of the programming side, there are multiple ways I could've improved it, but couldn't in the interest of time, and the project not necessarily requiring it to function.

For instance, my code was all put into one `main.cpp` file. This was purely out of convenience; however, it would've been good practise to sort parts of my program into its separate programs.

As I was the only one working on the code, I ended up not adding many comments, as I knew the logic behind my code. However, just for future reference for myself and any others, I should've added comments that explained my code.

In terms of the project management side, I learnt effective, applicable skills to be a leader. I set up infrastructure for the project to run smoothly, and most importantly, to stay organised.

For instance, the Check In page I created allowed us to track progress, and for people to update their individual progress in their own spreadsheet. However, this system relied on the autonomy of each person in the group, which unfortunately wasn't reflected enough to make the system useful.

I organised and scheduled weekly meetings, as well as communicating with Andy about our progress. These tasks helped me development my skills in time management and effective communication when it comes to a group project.

Peer Review

Dylan – 35 shares

- As previously stated, I took the roles of Project Lead, and Lead Programmer.
- I coded the entire game myself, as well as making code that was flexible and scalable, to make future expansions easier.
- I also undertook more roles further on into the project, such as Secretary, but more notably roles for the Tender Presentation Video deliverable, where I animated, sound designed, video edited all of it.

Marcy – 25 shares

- Marcy led the hardware portion of our product, including wiring up the inputs, and setting up the LCD screen, including the wiring and creating a class.
- She also was our Creative Lead and gave us the initial ideas and design choices early in the project's conceptions. She gave us detailed outlines of future implementations we could add to further the product beyond the prototype.
- As she had the most technical knowledge behind engineering a physical product, she led a lot of the hardware section by herself.

Jacob – 20 shares

- As PD was new software for all of us, Jacob was appointed to learn the software, and become our PD engineer.
- He was very successful in doing this, learning how PD works and was able to create a functioning PD patch from scratch. He set up the basis of the sound design and the audio part of our product, as well as implementing the MIDI-like communication between the code and PD.
- He also was future-thinking, adding all the parameters as specified in our Tender Presentation Video, and even a menu music system, showcasing the potential of the product.

Tyler – 12 shares

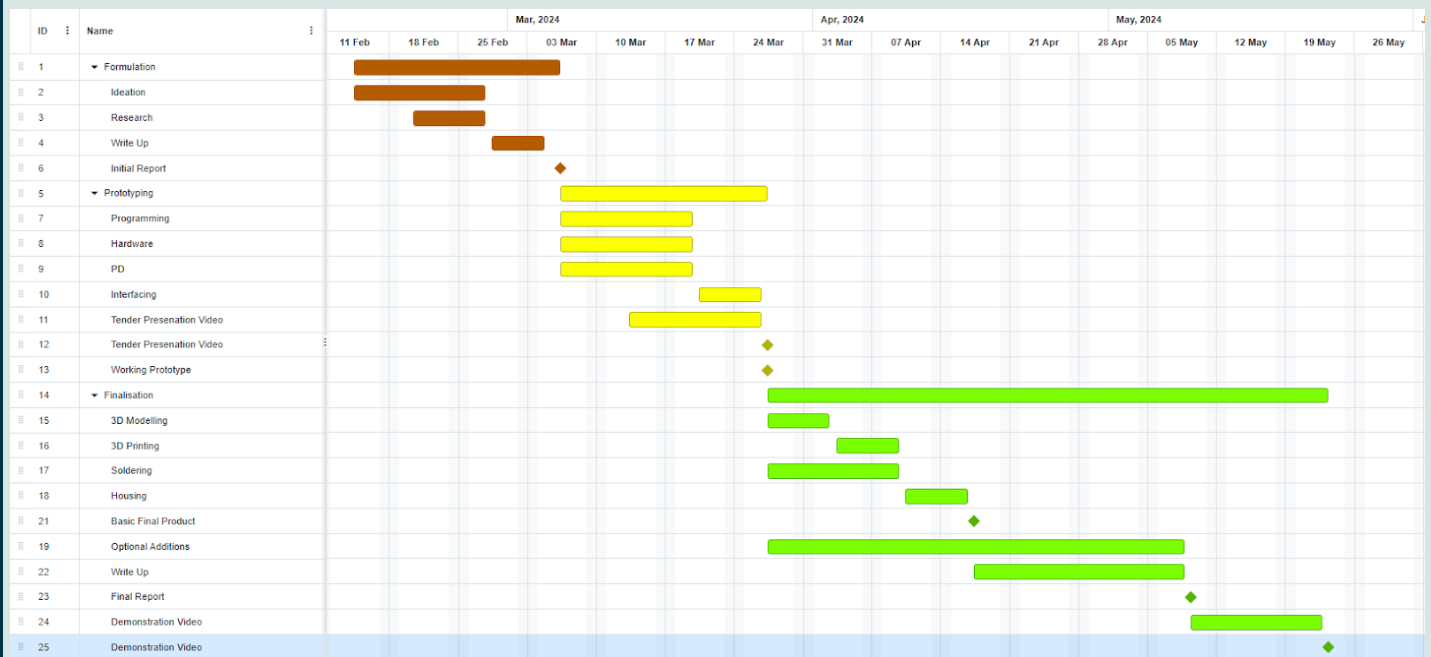
- Tyler was assigned to be secretary, treasurer, and 3D modeller.
- A week or so into the project starting, Tyler relinquished the secretary role to me, and I had the duty of taking meeting notes and organising our communication channels.
- He was able to fulfil his role as treasurer and 3D modeller, as was able to estimate and track our company spending.
- He created a comprehensive 3D model for the product, using AutoCAD. He was able to learn the software from scratch to create a casing that would house all the electronics.
- He successfully created an aesthetically pleasing product, while also minimising product risks as detailed in the Risk Assessment (*see Appendix C*)

Ben – 8 shares

- Initially, Ben was assigned to assist me with programming and quality testing.
- Unfortunately, he couldn't fulfil his responsibilities with programming, as I had to program all the code by myself. Even when given tasks, such as writing algorithms and researching ways to implement how the menus would work, he was not able to deliver.
- However, he did play a role in writing scripts and storyboards for our Tender Presentation Video, which proved valuable for our deliverables.
- For our Initial Report, he also delved deeper in the ideation stage, and did market research to find potential gaps in the market, or oversaturated ideas.

Appendix

Appendix A



Appendix B

GROUP CHECK IN								
WEEK	TASK TITLE	PRIORITY	BEN	DYLAN	JACOB	MARCY	TYLER	COMMENTS
1	Research I	HIGH	-	90	90	90	90	Research individual parts of the project.
1	Ideation I	MEDIUM	90	-	90	90	90	Come up with ideas, and narrow them down into one.
2	Research II	HIGH	90	-	90	90	90	Look into practical application of hardware, software and PD
2	Write-up I	LOW	90	-	-	-	90	Write up progress of the week to start arranging into first deliverable
3	Write-up II	HIGH	90	90	90	90	90	Start writing paragraphs for the Week 4 Deliverable
3	Programming I	MEDIUM	90	90	-	-	-	Implement basic logic and structures
3	PD I	MEDIUM	-	-	90	-	90	Create patches with adjustable parameters and create presets
3	Hardware I	MEDIUM	-	-	90	90	-	Create block designs and plans for inputs
4	Interfacing I	MEDIUM	90	90	90	90	90	Get code to change parameters on LCD screen, code to PD, and PD to sound
4	Presets I	LOW	-	-	90	-	90	Create workable presets for PD patches
4	Scripting I	MEDIUM	90	-	-	-	90	Make script for Week 7 tender presentation
5	Storyboarding I	HIGH	90	-	90	-	-	Decide on what visuals are going where in the tender presentation
5	Filming I	HIGH						Film the relevant parts of the video that must be done in person. Update: SCRAPPED, voiceovers instead

Appendix C

RISK ASSESSMENT

NAME	Y1S2 Music Tech Project				OBJECTIVE	Create a budget games console
TYPE OF RISK	PRE-MITIGATION				DEPARTMENT / LOCATION	MITIGATIONS / WARNINGS / REMEDIES
	RISK	RISK SEVERITY	RISK LIKELIHOOD	RISK LEVEL		
Functionality	Edges are sharp, inducing a potential cut	TOLERABLE	POSSIBLE	LOW	Modelling	Bevel edges in 3D model
	Components get loose in the printed body	UNDESIRABLE	POSSIBLE	MEDIUM	Construction	Solder electronic components to a PCB, and glue down any loose components to printed body
	Parameters are unclear on what they change	TOLERABLE	POSSIBLE	LOW	Modelling/Coding	Clear icons printed on body, and text on LCD screen explaining what is being changed.
	Game is too hard/not fun	UNDESIRABLE	PROBABLE	HIGH	Ideation	
Management	Group members unsure on what to do each week	INTOLERABLE	PROBABLE	HIGH	Management	Check-in page, outlining goals for the week and who's working on it. Creative and/or Project lead open to communication too.
	Group members not attending meetings	UNDESIRABLE	PROBABLE	MEDIUM	Management	Message absent members on progress, forward weekly meeting notes and check-in page.
Development	Project is too ambitious, not able to finish in time	UNDESIRABLE	POSSIBLE	HIGH	Ideation	Ideas critiqued by other members. Gantt chart created to track time and deadlines.
	Not enough work being done, not able to finish in time	UNDESIRABLE	POSSIBLE	HIGH	Development	Reduce scope of project. Gantt chart created to track time and deadlines.
						Online copy available on the cloud